

# Decoding Leela Chess Zero (LC0): Neural MCTS, Linear Probes & The PUCT Formula

Chess Speak Out Loud — Design Session Documentation

August 6, 2026

## Contents

|          |                                                        |          |
|----------|--------------------------------------------------------|----------|
| <b>1</b> | <b>Executive Summary</b>                               | <b>2</b> |
| <b>2</b> | <b>The LC0 Neural Architecture &amp; Linear Probes</b> | <b>2</b> |
| 2.1      | Engine Binary vs. Weight File . . . . .                | 2        |
| 2.2      | Transformer Layer Forward Pass . . . . .               | 2        |
| 2.3      | Linear Probes on Intermediate Layers . . . . .         | 2        |
| <b>3</b> | <b>Neural Monte Carlo Tree Search (Neural MCTS)</b>    | <b>3</b> |
| 3.1      | The 4-Phase Iterative Loop . . . . .                   | 3        |
| <b>4</b> | <b>The PUCT Formula: Rigorous Derivation</b>           | <b>3</b> |
| 4.1      | The Full Mathematical Expression . . . . .             | 3        |
| 4.2      | Definition of Terms . . . . .                          | 3        |
| 4.3      | Asymptotic Behavior . . . . .                          | 3        |
| <b>5</b> | <b>Real-World Analogy: The Job Interview Model</b>     | <b>4</b> |

# 1 Executive Summary

This document provides a rigorous mathematical and conceptual breakdown of the internal mechanics of Leela Chess Zero (LC0). It covers:

1. The architecture of LC0’s 15-layer Transformer network (BT3-768x15x24h).
  2. How **Linear Probes** read intermediate activations and future look-ahead.
  3. The step-by-step **Search-Inference Loop** during Monte Carlo Tree Search (MCTS).
  4. The complete derivation and history of the **PUCT** formula.
  5. Real-world intuitive analogies (Slot Machines, Dating, and Job Interviews) explaining the exploration-exploitation balance.
- 

## 2 The LC0 Neural Architecture & Linear Probes

### 2.1 Engine Binary vs. Weight File

The lc0 system consists of two distinct components:

- **C++ Engine Binary:** Manages Monte Carlo Tree Search (MCTS), node visit counts, move ordering, and GPU batching.
- **Neural Weights File (.pb.gz / .onnx):** A 15-layer Transformer network (BT3-768x15x24h) that outputs move instincts and position evaluations.

### 2.2 Transformer Layer Forward Pass

When a position  $s$  (represented as 64 spatial board tokens of dimension  $d = 768$ ) enters the network, it passes through 15 sequential encoder layers:

$$\mathbf{x}^{(0)} \xrightarrow{\text{Layer 1}} \mathbf{x}^{(1)} \xrightarrow{\text{Layer 2}} \dots \xrightarrow{\text{Layer 8}} \dots \xrightarrow{\text{Layer 14}} (P(a|s), V(s)) \quad (1)$$

Where:

- $P(a|s) \in [0, 1]^{1858}$  is the **Policy Head** (prior probability over legal move candidates).
- $V(s) = (W, D, L) \in [0, 1]^3$  is the **Value Head** (Win/Draw/Loss distribution).

### 2.3 Linear Probes on Intermediate Layers

Research demonstrates that middle layers (Layers 6–10) construct internal representations of the board state 3 to 7 plies ahead. A **Linear Probe** is a classifier  $\mathbf{W}_{\text{probe}}$  trained directly on the activation tensor  $\mathbf{x}^{(k)}$  at layer  $k$ :

$$\hat{\mathbf{y}}_{\text{future}} = \sigma \left( \mathbf{W}_{\text{probe}} \mathbf{x}^{(k)} + \mathbf{b} \right) \quad (2)$$

This allows our diagnostic system to detect:

- **Foreseen Future Board States:** Extracting piece locations 4 plies ahead before full MCTS expansion.
  - **Suppressed Win Probe (The "Tal Probe"):** Detecting positions where intermediate layers  $\mathbf{x}^{(8)}$  identify a winning sacrificial continuation, but output layer  $P(a|s)$  suppresses it due to high material penalty priors.
-

### 3 Neural Monte Carlo Tree Search (Neural MCTS)

#### 3.1 The 4-Phase Iterative Loop

During analysis, the engine executes an iterative 4-phase loop:

1. **Selection:** Traverses the existing tree from the root using the **PUCT** formula to select the leaf node  $s$ .
2. **Expansion:** Creates a child node for a selected candidate move  $a$ .
3. **Neural Inference:** Submits the new child board  $s'$  to the GPU for a forward pass, obtaining  $P(a'|s')$  and  $V(s')$ .
4. **Backpropagation:** Updates total visit counts  $N(s, a)$  and cumulative win evaluations  $W(s, a)$ , yielding updated expected value  $Q(s, a) = W(s, a)/N(s, a)$ .

—

### 4 The PUCT Formula: Rigorous Derivation

#### 4.1 The Full Mathematical Expression

At node  $s$ , the selection rule picks the action  $a^*$  maximizing the PUCT objective:

$$a^* = \arg \max_a [Q(s, a) + U(s, a)] \quad (3)$$

Where the Upper Confidence Bound term  $U(s, a)$  is explicitly defined as:

$$U(s, a) = c_{\text{puct}} \cdot P(a|s) \cdot \frac{\sqrt{\sum_b N(s, b)}}{1 + N(s, a)} \quad (4)$$

#### 4.2 Definition of Terms

| Symbol            | Mathematical Definition                         | Role in Search                                       |
|-------------------|-------------------------------------------------|------------------------------------------------------|
| $Q(s, a)$         | $\frac{W(s, a)}{N(s, a)}$ (Mean Action Value)   | <b>Exploitation:</b> Real calculated win rate        |
| $P(a s)$          | Policy Prior Probability                        | <b>Intuition:</b> Neural Network’s 0-node instinct   |
| $N(s, a)$         | Visit Count for move $a$                        | <b>Experience:</b> Number of times line evaluated    |
| $\sum_b N(s, b)$  | Parent Node Total Visits                        | <b>Context:</b> Total depth budget spent at node $s$ |
| $c_{\text{puct}}$ | Exploration Constant ( $\approx 1.25$ – $2.5$ ) | Scales policy influence relative to $Q$              |

Table 1: Components of the PUCT Formula

#### 4.3 Asymptotic Behavior

- **Low Visit Regime** ( $N(s, a) \rightarrow 0$ ):  $U(s, a) \gg Q(s, a)$ . Search priority is almost entirely dominated by the neural prior  $P(a|s)$ . High-policy moves are calculated first.
- **High Visit Regime** ( $N(s, a) \rightarrow \infty$ ):  $U(s, a) \rightarrow 0$ . Search priority converges to pure action value  $Q(s, a)$ . The calculation reflects true tactical outcome.
- **Refutation Mechanism:** If a high-policy move  $a_{\text{natural}}$  blunders materially,  $Q(s, a_{\text{natural}})$  drops to 0. As a result, its priority collapses, forcing  $U(s, a_{\text{alt}})$  for lower-policy moves to exceed it, automatically triggering a search pivot.

—

## 5 Real-World Analogy: The Job Interview Model

To understand how PUCT optimizes search resource allocation, consider a hiring manager filling 1 job position from 100 applicants:

$$\text{Interview Priority} = \underbrace{Q(\text{Interview Score})}_{\text{Exploitation of Verified Ability}} + c_{\text{puct}} \cdot \underbrace{P(\text{Resume Score})}_{\text{Policy Prior Prediction}} \cdot \underbrace{\frac{\sqrt{\text{Total Interviews Done}}}{1 + \text{Rounds with Candidate}}}_{\text{Uncertainty Bonus}} \quad (5)$$

- **$P(a|s)$  (Resume Rating):** HR scans resumes before interviewing. Candidate A (ex-Google, 10 yrs exp) has  $P_A = 70\%$ . Candidate B has  $P_B = 20\%$ . Candidate A gets Round 1 first.
- **$Q(s, a)$  (Technical Coding Performance):** Candidate A performs well in Round 1 ( $Q_A = 0.95$ ). Because  $Q_A$  is high, candidate A is invited to Rounds 2 and 3.
- **Refutation (Bombing Round 3):** In Round 3, Candidate A fails System Design ( $Q_A$  drops to 0.10).
- **The Pivot:** Candidate B has 0 rounds done ( $N_B = 0$ ), so Candidate B's Uncertainty Bonus exceeds Candidate A's decayed score ( $Q_A = 0.10$ ). HR immediately pivots to interview Candidate B.